

# Educação Profissional Paulista

Técnico em  
**Ciência de  
Dados**

# **Bibliotecas: Pandas, NumPy, SciPy, Matplotlib e Seaborn**

## **Matplotlib: estrutura**

Aula 3

Código da aula: [DADOS]ANO1C2B4S29A3

**Bibliotecas: Pandas,  
NumPy, SciPy,  
Matplotlib e Seaborn**

## **Mapa da Unidade 5 Componente 3**

**Você está aqui!**  
Matplotlib: estrutura

*semana*

**29**

Matplotlib:  
gráficos básicos

*semana*

**30**

*semana*

**23**

Pandas: acesso  
e seleção

*semana*

**28**

Pandas:  
visualização gráfica

**Bibliotecas: Pandas,  
NumPy, SciPy,  
Matplotlib e Seaborn**

## **Mapa da Unidade 5 Componente 3**

# **Você está aqui!**

**29**

**Matplotlib: estrutura**

**Aula 3**

Código da aula:  
[DADOS]ANO1C2B4S29A3



## Objetivos da aula

- Conhecer as estruturas da figura do Matplotlib, criar múltiplos gráficos na mesma figura e salvar os gráficos.



## Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



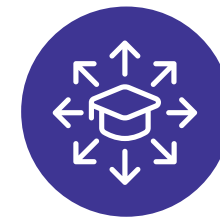
## Duração da aula

50 minutos.



## Competências técnicas

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Usar técnicas para explorar e analisar dados, aplicar modelos estatísticos, identificar padrões, realizar inferências e tomar decisões com base em evidências.



## Competências socioemocionais

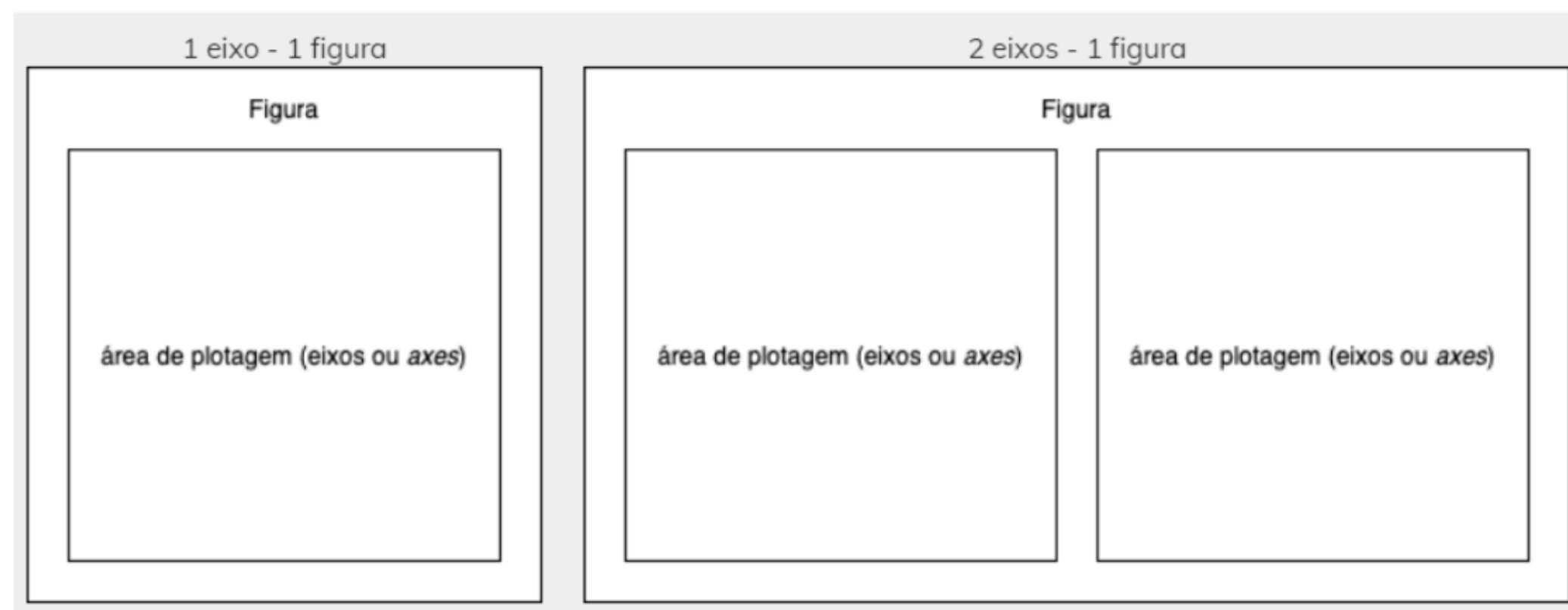
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.

# Construindo o **conceito**

## Matplotlib

Na estrutura do Matplotlib temos dois conceitos importantes:

- ▶ Área de plotagem (eixos ou Axes) onde de fato os gráficos aparecem;
- ▶ Figura (Figure) onde os eixos estão.



Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

## Construindo o conceito

# Matplotlib

### Conheça cada característica de Figure, Axes e Axis:

**Figure:** A figura inteira. A Figura acompanha todas as crianças Axes, um grupo de Artistas “especiais” (como títulos, legendas de figuras, barras de cores etc.) e até mesmo subfiguras aninhadas. Você pode criar uma nova Figura usando o `pyplot.figure()`.

**Axes:** Um Axes é uma arte (Artist) anexada a uma Figura que contém uma região para plotagem de dados. Cada Axes também tem um título, rótulo x e rótulo y. Você pode criar uma figura com eixos usando `pyplot.subplots()`.

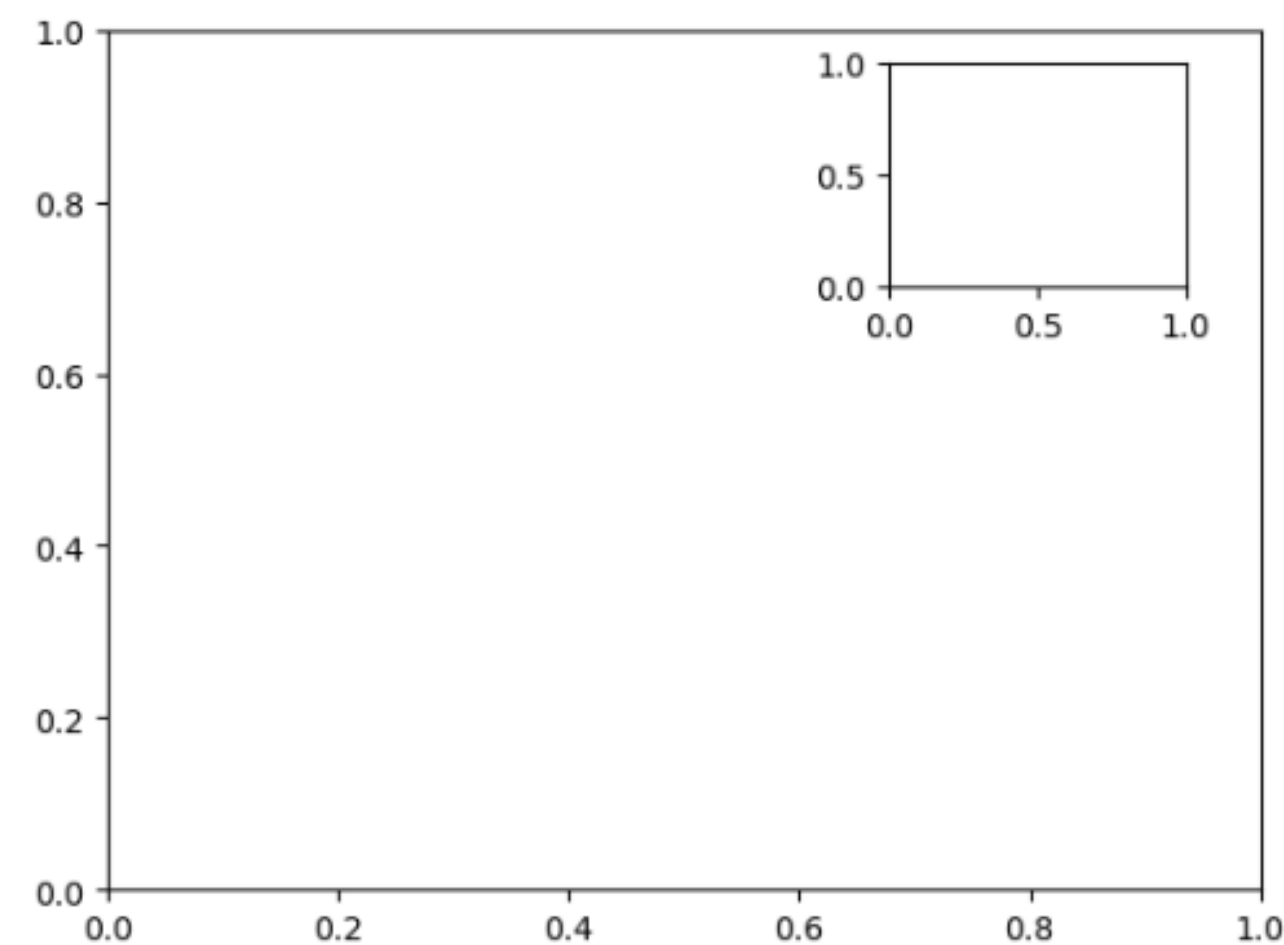
**Axis:** Esses objetos definem a escala, os limites e geram ticks (as marcas no Axis) e ticklabels (strings que rotulam os ticks). A combinação correta de Locator e Formatter fornece controle preciso sobre os locais e rótulos dos ticks.

Construindo  
o **conceito**

## Subplots

Definindo a área de plotagem.  
Alternativa 1: Manualmente.

```
1 # Definindo a área de plotagem  
2 ax1 = plt.axes()  
3 ax2 = plt.axes([0.65, 0.65, 0.2, 0.2])
```



Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

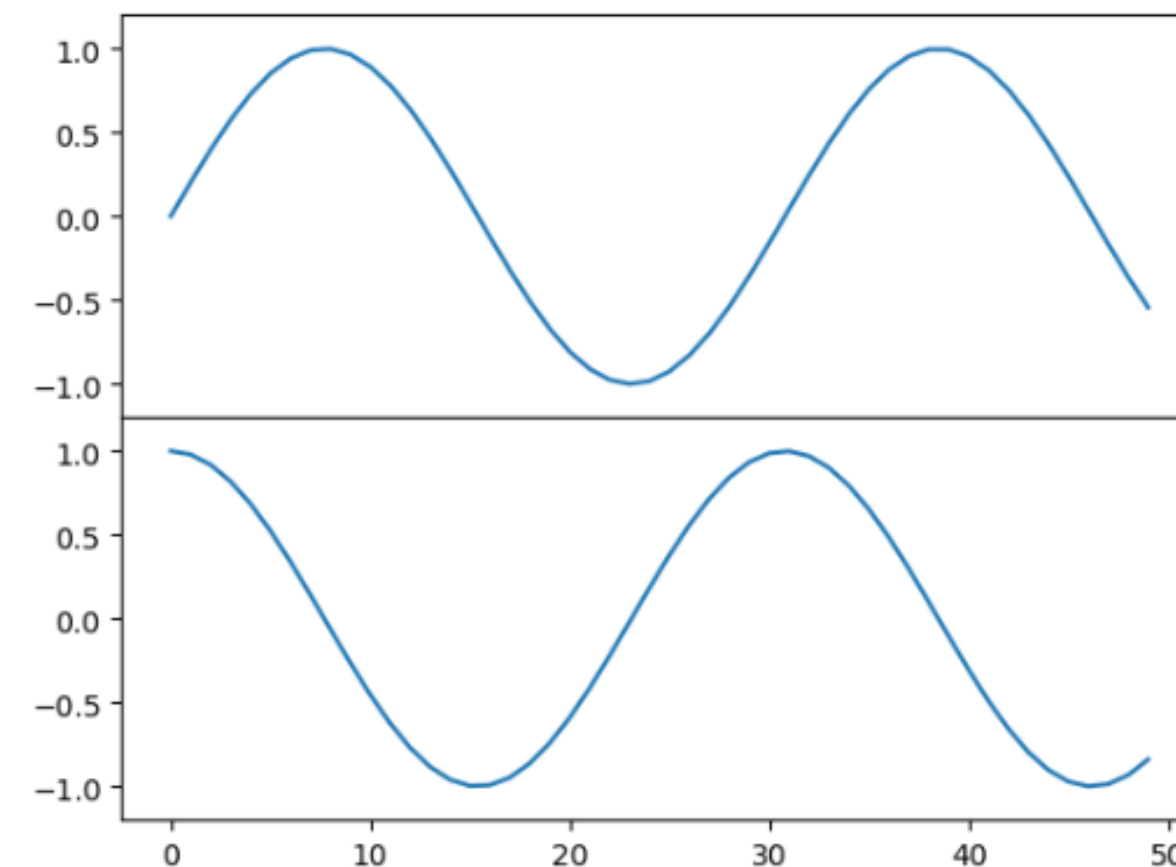
# Construindo o conceito

## Subplots

Definindo a área de plotagem.

**Alternativa 2:** Criar a área de plotagem e, logo após, adicionar os gráficos.

```
1 fig = plt.figure()
2 ax1 = fig.add_axes([0.1, 0.5, 0.8, 0.4], xticklabels=[], ylim=(-1.2, 1.2))
3 ax2 = fig.add_axes([0.1, 0.1, 0.8, 0.4], ylim=(-1.2, 1.2))
4
5 x2 = np.linspace(0, 10)
6
7 ax1.plot(np.sin(x2))
8 ax2.plot(np.cos(x2))
9 plt.show()
```



Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

# Construindo o conceito

## Subplots

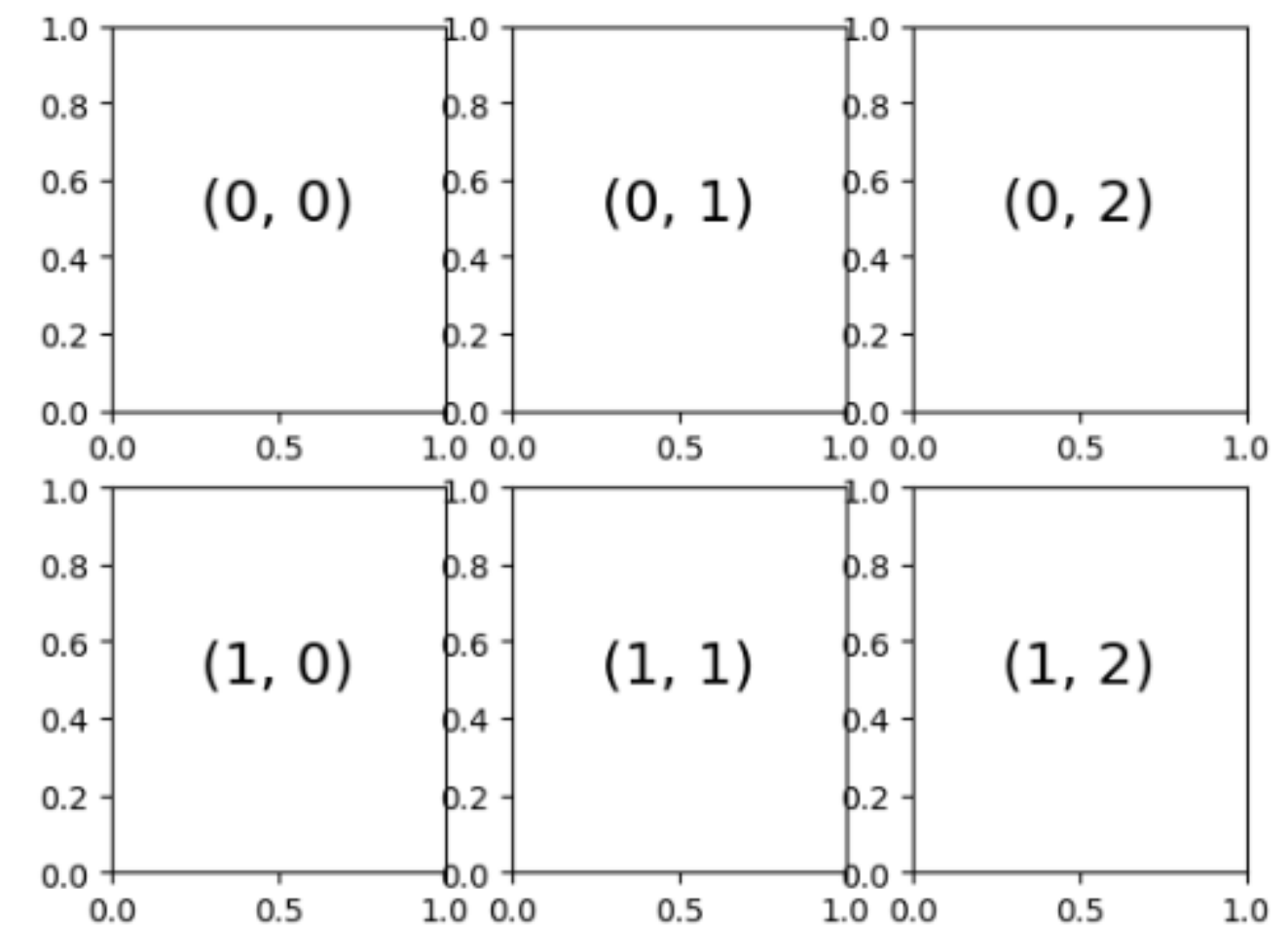
Usando o `plt.subplots()`:

`plt.subplot(nrows, ncols)`

`nrows` = nº de linhas

`ncols` = nº de colunas

```
1 fig, ax = plt.subplots(nrows=2, ncols=3)
2
3
4 # Identificando cada figura -> didático
5 for i in range(2):
6     for j in range(3):
7         ax[i, j].text(0.5, 0.5, str((i, j)),
8                       fontsize=18, ha='center')
```



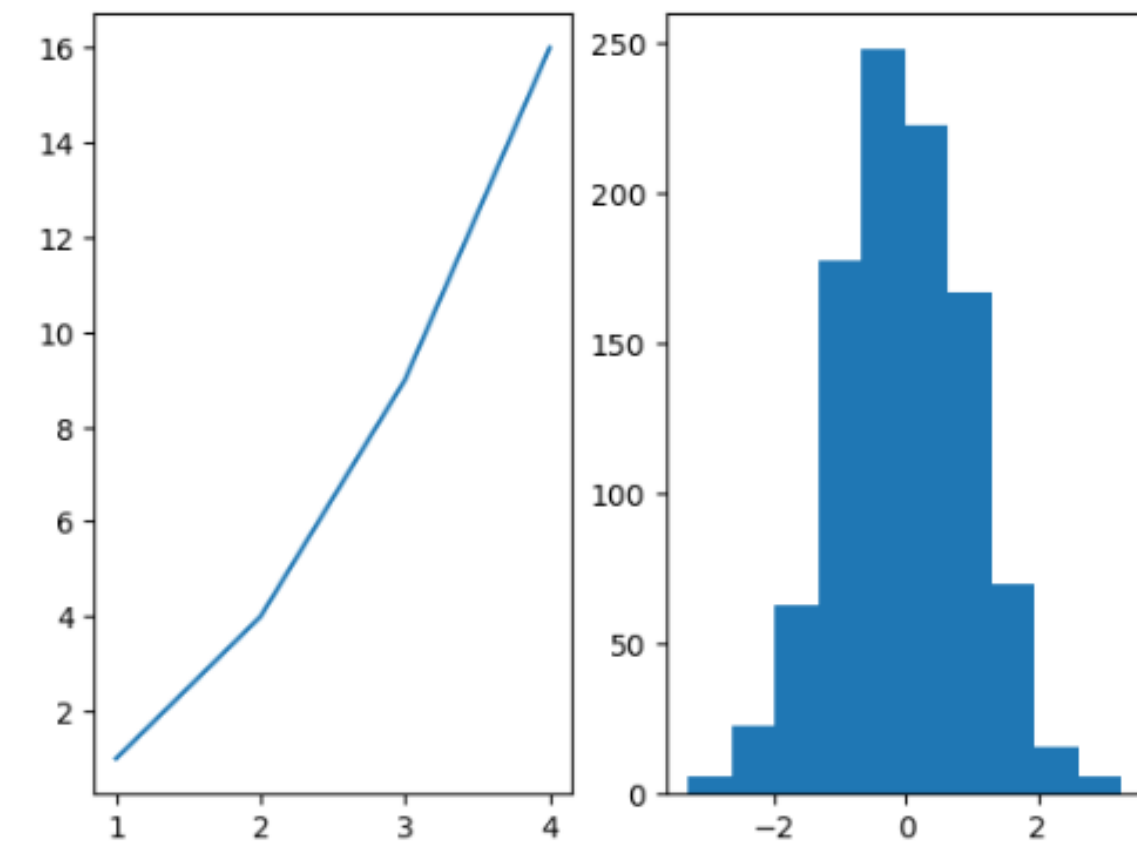
Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo  
o **conceito**

## Subplots

Criando dois gráficos em uma linha.

```
1 # criando a estrutura
2
3 fig, axes = plt.subplots(1, 2)
4
5 # criar os gráficos em cada posição
6 x = [1, 2, 3, 4]
7 y = [1, 4, 9, 16]
8 z = np.random.randn(1000)
9 t = [3, 6, 12, 20]
10
11 axes[0].plot(x, y)
12 axes[1].hist(z);
```



Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

## Construindo o **conceito**

# Salvando o gráfico

Para salvar um gráfico criado com o Matplotlib no Jupyter Notebook, você pode seguir estas opções:

### ► 1) Chamar savefig antes de show:

```
1 plt.savefig('meu_grafico.png', format='png')  
2 plt.show()
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Isso garante que a figura seja salva antes de ser zerada para preparar outro gráfico.

## Construindo o **conceito**

# Salvando o gráfico

- ▶ 2) Utilizar `plt.gcf()` (de “get current figure”) para salvar o gráfico em uma variável e salvá-la a qualquer momento:

```
1 fig = plt.gcf()
2 plt.show()
3 fig.savefig('meu_outro_grafico.png', format='png')
```

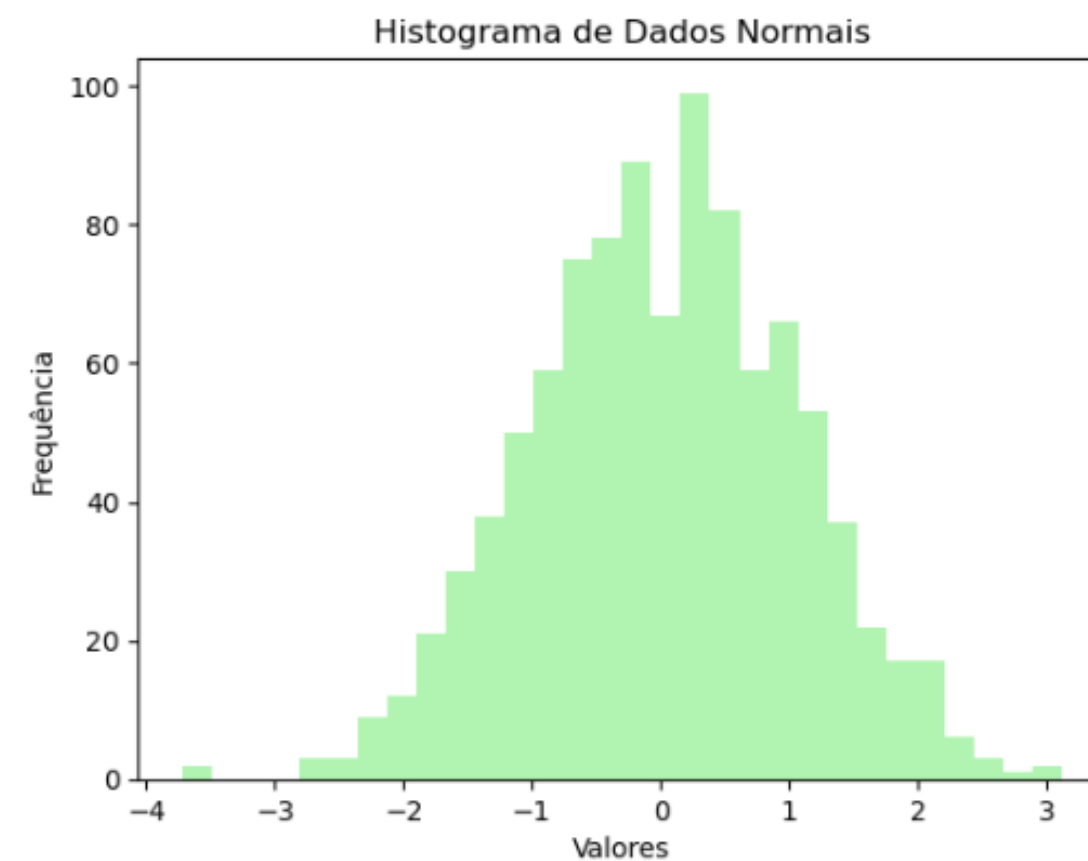
Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Desta forma, você pode controlar quando salvar a figura e evitar que a legenda seja cortada.

Construindo  
o **conceito**

## Salvando o gráfico – Exemplos

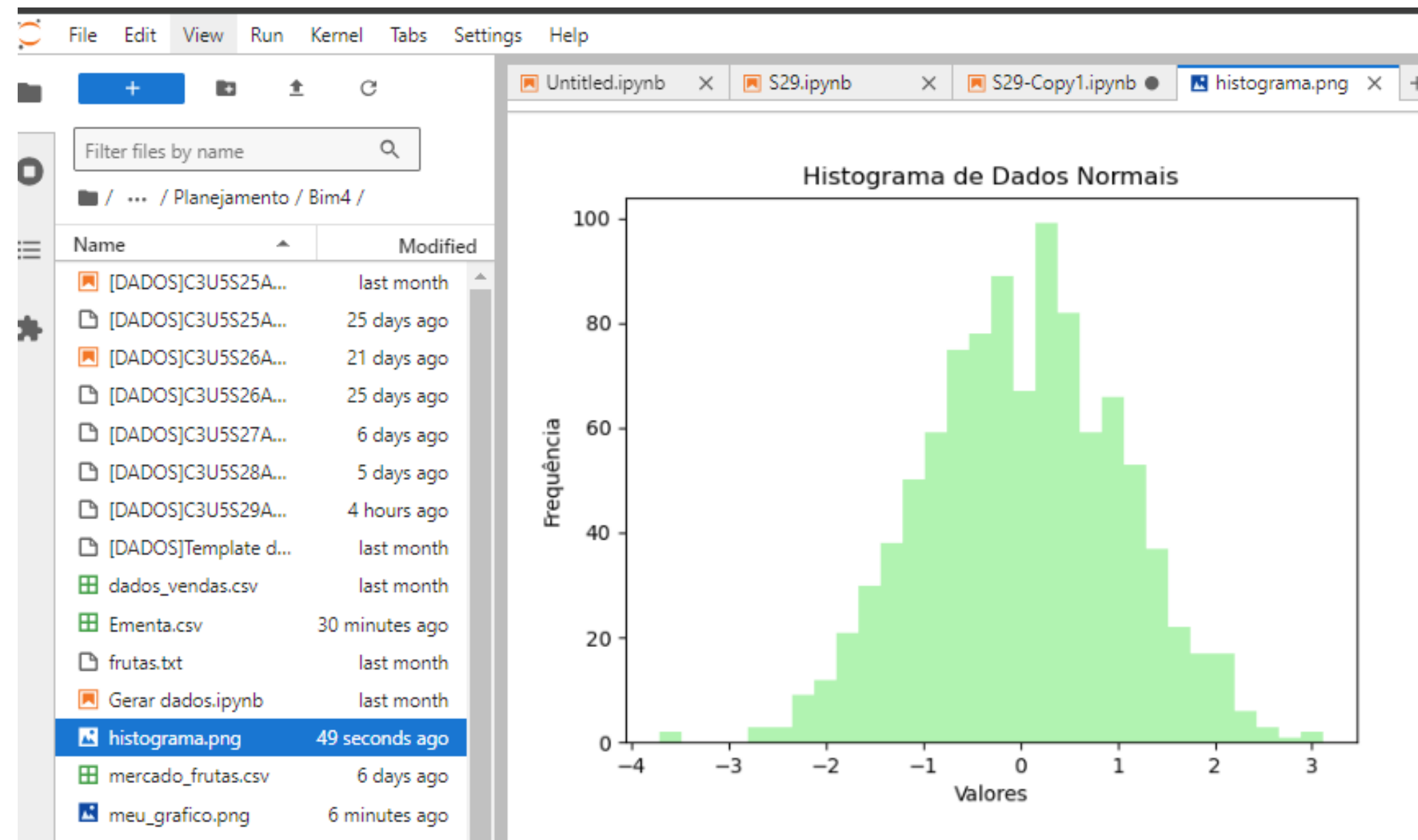
```
1 dados = np.random.normal(0, 1, 1000)
2
3 # Criando o histograma
4 plt.hist(dados, bins=30, color='lightgreen', alpha=0.7)
5 plt.title('Histograma de Dados Normais')
6 plt.xlabel('Valores')
7 plt.ylabel('Frequência')
8
9 plt.savefig('histograma.png', format='png')
10 plt.show()
```



Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo  
o **conceito**

## Salvando o gráfico – Exemplos

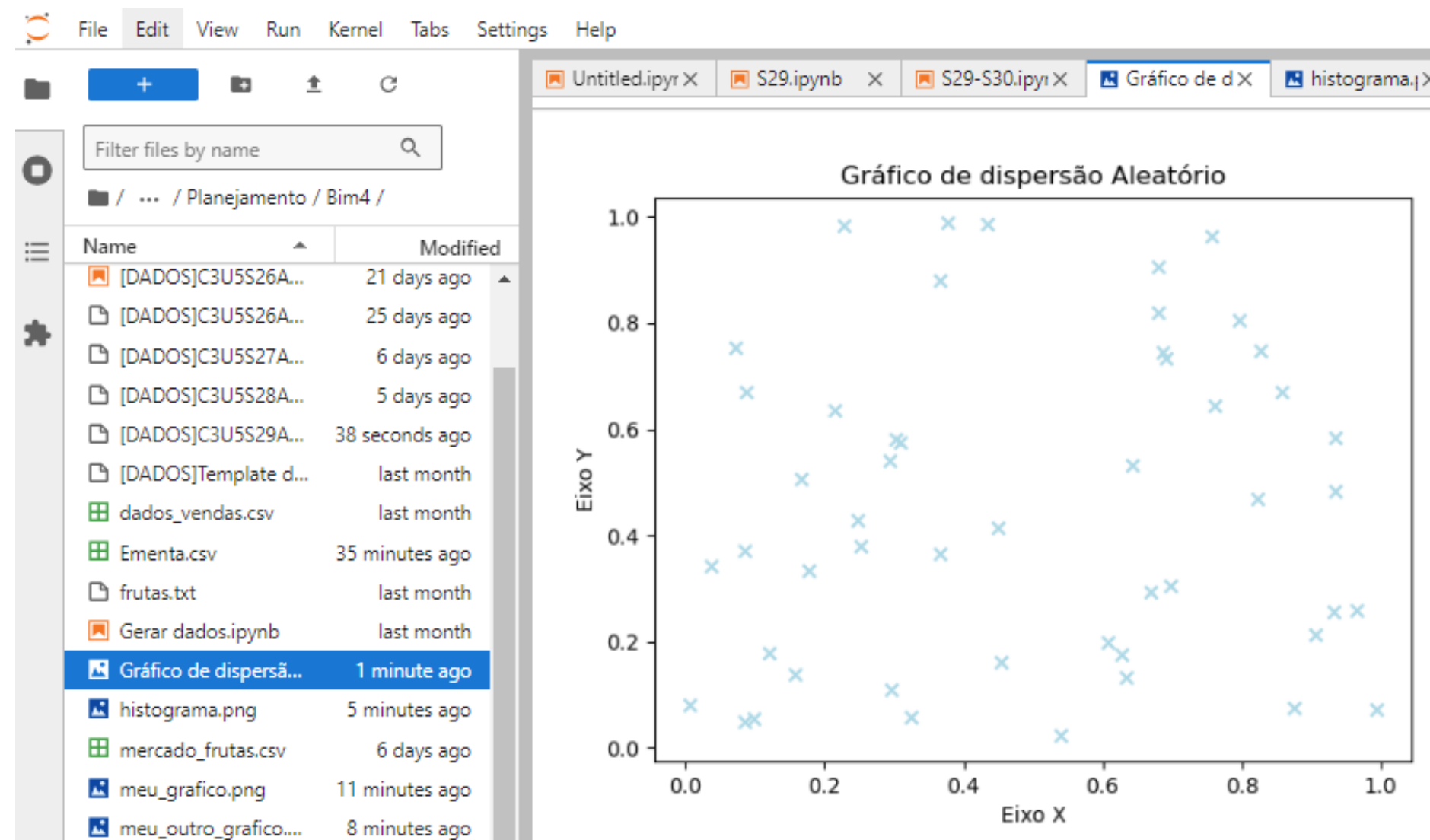
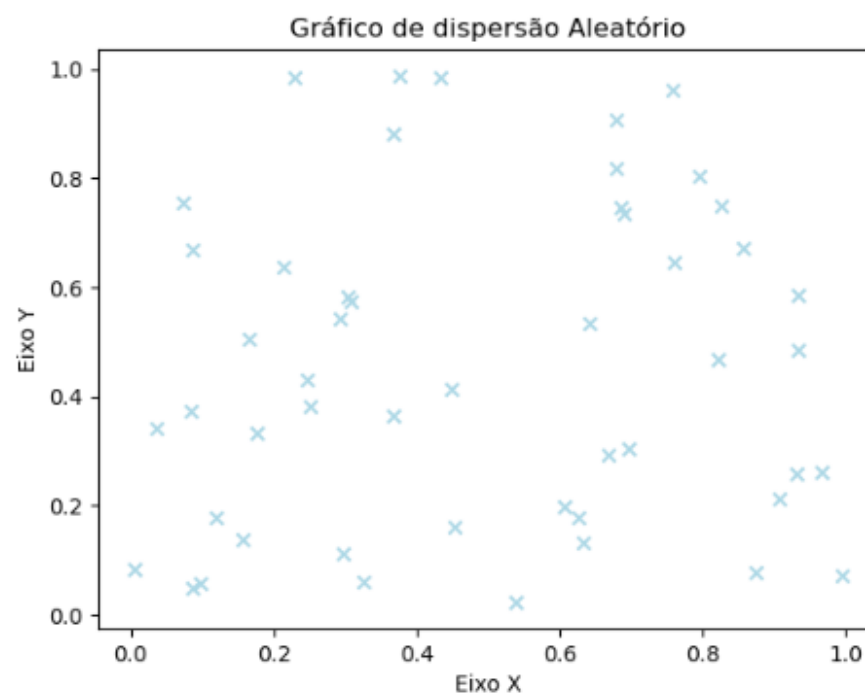


Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo  
o **conceito**

## Salvando o gráfico – Exemplos

```
1 fig = plt.gcf()
2
3 x = np.random.rand(50)
4 y = np.random.rand(50)
5
6 plt.scatter(x, y, color='lightblue', marker='x')
7 plt.title('Gráfico de dispersão Aleatório')
8 plt.xlabel('Eixo X')
9 plt.ylabel('Eixo Y')
10 plt.show()
11
12 fig.savefig('Gráfico de dispersão.png', format='png')
```



Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



## Colocando em **prática**

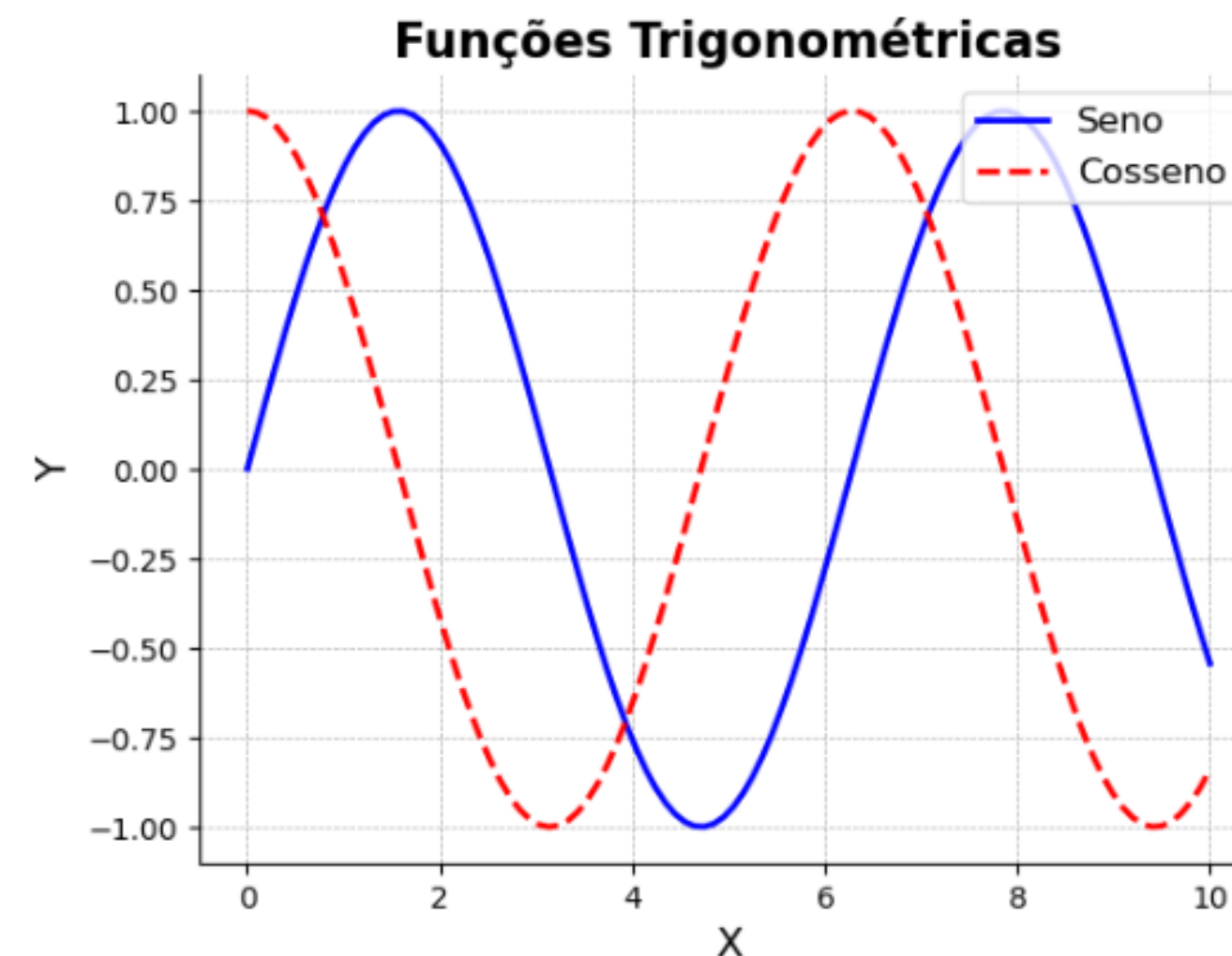
# Modificando o gráfico

Agora, vamos praticar a modificação. Observe o código abaixo e seu respectivo gráfico, e siga o passo a passo no próximo slide.

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Dados fictícios para o gráfico
5 x = np.linspace(0, 10, 100)
6 y1 = np.sin(x)
7 y2 = np.cos(x)
8
9 # Criando a figura e o eixo
10 fig, ax = plt.subplots()
11
12 # Adicionando as linhas do gráfico
13 ax.plot(x, y1, label='Seno', color='b', linewidth=2, linestyle='-')
14 ax.plot(x, y2, label='Cosseno', color='r', linewidth=2, linestyle='--')
15
16 # Adicionando título e rótulos dos eixos
17 ax.set_title('Funções Trigonômétricas', fontsize=16, fontweight='bold')
18 ax.set_xlabel('X', fontsize=14)
19 ax.set_ylabel('Y', fontsize=14)
20
21 # Adicionando uma grade
22 ax.grid(True, which='both', linestyle='--', linewidth=0.5)
23
24 # Personalizando os eixos
25 ax.spines['top'].set_visible(False)
26 ax.spines['right'].set_visible(False)
27
28 # Adicionando a legenda
29 ax.legend(loc='upper right', fontsize=12)
30
31 # Exibindo o gráfico
32 plt.show()

```



Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



**Durante a aula.**



**Em grupo de até quatro alunos.**



**Envio ao AVA**



Colocando  
em **prática**

## Modificando o gráfico



**Durante a aula.**



**Em grupo de até  
quatro alunos.**



**Envio ao AVA**



### **ORGANIZEM-SE EM GRUPOS**

Reúnam-se em grupos de quatro alunos.



### **OBSERVE E ANALISE**

Analise o código e a imagem gerada apresentados no slide anterior.



### **MOMENTO DA AÇÃO**

Modifiquem a imagem com os seguinte pontos:

- O título na cor verde;
- Os rótulos dos eixos devem ser "Eixo X" e "Eixo Y";
- Deve haver uma linha amarela pontilhada em  $X = 4$ ;
- As marcações do eixo y devem estar rotacionadas em 90 graus.



### **ENTREGA**

Envie o arquivo em **.ipynb** no AVA.



© Getty Images

O que nós  
**aprendemos  
hoje?**

## Então ficamos assim...

- 1** Entendemos a diferença entre figura e área de plotagem.
- 2** Conhecemos subplots, onde podemos desenhar vários gráficos na mesma área de plotagem.
- 3** Aprendemos como é realizada a ação de salvar um gráfico/figura.

# Saiba mais

Entenda o funcionamento de figuras e eixos do Matplotlib. Customize suas visualizações com diferentes cores, estilos, anotações, e muito mais!

ALURA. *Data Science. Data Visualization*: criação de gráficos com o Matplotlib, 10 abr. 2023. Disponível em: <https://www.alura.com.br/curso-online-customizacao-matplot>. Acesso em: 6 ago. 2024.

# Referências da aula

Identidade visual: Imagens © Getty Images.

MATPLOTLIB. *Matplotlib 3.9.2 documentation*, [s.d.]. Disponível em: <https://matplotlib.org/stable/>. Acesso em: 6 ago. 2024.

MCKINNEY, W. *Python para análise de dados: tratamento de dados com Pandas, NumPy & Jupyter*. São Paulo: Novatec, 2023.

# Educação Profissional Paulista

Técnico em  
**Ciência de  
Dados**